Practical No:3

3. Implement Min, Max, Sum and Average operations using Parallel Reduction.

Code:

```
#include <iostream>
#include <omp.h>
#include <cstdlib>
#include <ctime>
using namespace std;

int main() {
    int n = 10000000;
   int *arr = new int[n];

   srand(time(0));
   for (int i = 0; i < n; i++)
      arr[i] = rand() % 100;

   // ---------- Sequential ----------
   int min_val = arr[0], max_val = arr[0];
   long long sum = 0;

   double start = omp_get_wtime();

   for (int i = 0; i < n; i++) {
      if (arr[i] < min_val)
         min_val = arr[i];
      if (arr[i] > max_val)
         max_val = arr[i];
      sum += arr[i];
   }

   double end = omp_get_wtime();

   cout << "\n--- Sequential Results ---";
   cout << "\nMin = " << min_val;
   cout << "\nMax = " << max_val;
   cout << "\nSum = " << sum;
   cout << "\nAverage = " << (double)sum / n;
```

```
    cout << "\nTime = " << (end - start) << " sec";

    // ---------- Parallel ----------
    int p_min = arr[0], p_max = arr[0];
    long long p_sum = 0;

    start = omp_get_wtime();

    #pragma omp parallel for reduction(+:p_sum) reduction(min:p_min) reduction(max:p_max)
    for (int i = 0; i < n; i++) {
        p_sum += arr[i];

        if (arr[i] < p_min)
            p_min = arr[i];

        if (arr[i] > p_max)
            p_max = arr[i];
    }

    end = omp_get_wtime();

    cout << "\n\n--- Parallel Results ---";
    cout << "\nMin = " << p_min;
    cout << "\nMax = " << p_max;
    cout << "\nSum = " << p_sum;
    cout << "\nAverage = " << (double)p_sum / n;
    cout << "\nTime = " << (end - start) << " sec";

    delete[] arr;

    return 0;
}
```

```
PS C:\Users\Rutuja Varunkar\Downloads\hpc> g++ -fopenmp P3.cpp -o P3
PS C:\Users\Rutuja Varunkar\Downloads\hpc> ./P3

--- Sequential Results ---
Min = 0
Max = 99
Sum = 494753458
Average = 49.4753
Time = 0.017 sec

--- Parallel Results ---
Min = 0
Max = 99
Sum = 494753458
Average = 49.4753
Time = 0.0119998 sec
PS C:\Users\Rutuja Varunkar\Downloads\hpc>
```